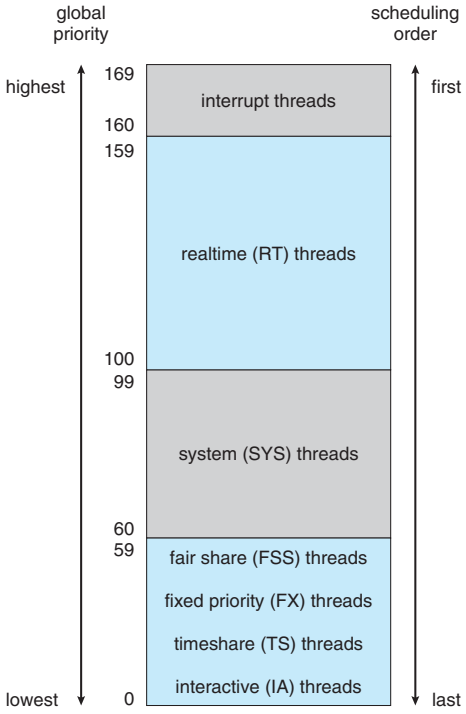
اولین مسأله، تعریف معیارهایی است که در گزینش یک الگوریتم مد نظر است. همان‌گونه که در بخش ۵-۲ دیدیم، معیارها در بیشتر موارد با تعابیری از بهره‌وری پردازنده، زمان پاسخ یا بازده تعریف می‌شوند. در انتخاب یک الگوریتم، نخست بایستی اهمیت نسبی این پارامترها را مشخص کنیم. معیارهای ما ممکن است شامل پارامترهای متعددی باشند که از آن جمله، می‌توان به موارد زیر اشاره کرد:

- بیشینه‌کردن بهره‌وری پردازنده با این محدودیت که حداکثر زمان پاسخ ۳۰۰ میلی‌ثانیه باشد.
- بیشینه‌کردن بازده، طوری که زمان گردش (در حالت متوسط) با زمان اجرای کل، تناسب خطی داشته باشد.

همین که معیارهای انتخاب تعریف شدند، می‌خواهیم الگوریتم‌های موردنظر را ارزیابی کنیم. در ادامه، روش‌های مختلف ارزیابی را که می‌توانیم استفاده کنیم، مطرح خواهیم کرد.



شکل ۵-۳۰ زمانبندی سولاریس.

۵-۸-۱ مدل‌سازی معین

یکی از رده‌های مهم روش‌های ارزیابی، **ارزیابی تحلیلی** است. ارزیابی تحلیلی با داشتن الگوریتم و بارکاری سیستم، فرمول یا عددی تولید می‌کند که کارایی الگوریتم را برای آن بارکاری ارزیابی می‌کند.

مدل‌سازی معین، یک نوع ارزیابی تحلیلی است. این روش، یک بارکاری خاص از پیش‌تعریف شده را می‌گیرد و کارایی هر یک از الگوریتم‌ها را برای آن بارکاری تعریف می‌کند. برای مثال، فرض کنید که بارکاری نشان داده شده در زیر را داشته باشیم. تمامی پنج پردازه با ترتیبی که مشخص شده‌اند، در لحظه‌ی ۰، با طول انفجار پردازنده برحسب میلی‌ثانیه می‌رسند:

پردازه	زمان انفجار
P_1	۱۰
P_2	۲۹
P_3	۳
P_4	۷
P_5	۱۲

الگوریتم‌های زمانبندی FCFS، SJF و RR (با کمیت ۱۰ میلی‌ثانیه) را برای این مجموعه از پردازه‌ها اعمال کنید. کدام الگوریتم، کمترین میانگین زمان انتظار را خواهد داد؟

با الگوریتم FCFS، پردازه‌ها را به صورت زیر اجرا خواهیم کرد:

P_1	P_2	P_3	P_4	P_5
۰	۱۰	۳۹	۴۲	۴۹ ۶۱

زمان انتظار پردازه‌ی P_1، ۰، P_2، ۱۰، P_3، ۳۹، P_4، ۴۲ و P_5، ۴۹ میلی‌ثانیه است. لذا، میانگین زمان انتظار برابر $28 = 5/(49 + 42 + 39 + 10 + 0)$ میلی‌ثانیه می‌شود.

با زمانبندی SJF غیر قابل پس‌گیر، پردازه‌ها را به صورت زیر اجرا می‌کنیم:

P_3	P_4	P_1	P_5	P_2
۰	۳	۱۰	۲۰	۳۲ ۶۱

زمان انتظار پردازه‌ی P_1، ۱۰، P_2، ۳۲، P_3، ۰، P_4، ۳ و P_5، ۲۰ میلی‌ثانیه است. بنابراین، میانگین زمان انتظار برابر $13 = 5/(10 + 32 + 0 + 3 + 20)$ میلی‌ثانیه می‌شود.

با الگوریتم RR، پردازه‌ها را به صورت زیر اجرا می‌کنیم:

P_1	P_2	P_3	P_4	P_5	P_2	P_5	P_2
۰	۱۰	۲۰	۲۳	۳۰	۴۰	۵۰ ۵۲	۶۱

زمان انتظار پردازه‌ی P_1، ۰، P_2، ۳۲، P_3، ۲۰، P_4، ۲۳ و P_5، ۴۰ میلی‌ثانیه است. بنابراین، میانگین زمان انتظار برابر $23 = 5/(0 + 32 + 20 + 23 + 40)$ میلی‌ثانیه خواهد بود.

در *این مورد* مشاهده می‌کنیم که میانگین زمان انتظار سیاست SJF، از نصف زمانبندی FCFS کمتر است، در حالی که الگوریتم RR یک مقدار متوسط به ما می‌دهد.

مدل‌سازی معین ساده و سریع است. اعداد دقیقی به‌دست می‌آوریم که امکان مقایسه‌ی الگوریتم‌ها را برای ما فراهم می‌کند. با این وجود، اعداد دقیقی به عنوان ورودی لازم دارد و جواب‌هایی هم که می‌دهد تنها در قبال آن حالت‌هاست. کاربرد اصلی مدل‌سازی معین در تشریح

الگوریتم‌های زمانبندی و ارائه‌ی مثال می‌باشد. در مواردی که برنامه‌ای را به دفعات اجرا می‌کنیم و می‌توان نیازهای پردازشی برنامه را به دقت اندازه‌گیری کرد، می‌توانیم برای انتخاب الگوریتم زمانبندی از مدل‌سازی معین بهره بگیریم. به علاوه، با مدل‌سازی معین می‌توان از میان مجموعه‌ای از نمونه‌ها، مواردی را که بیشتر به خواسته‌های ما نزدیک هستند مشخص کرده و جداگانه تحلیل و اثبات کرد. برای مثال، برای محیطی که مطرح شد (با مهیا بودن تمامی پردازه‌ها و انفجارهای آنها در لحظه‌ی ۰) می‌توان نشان داد که زمان انتظار پردازه‌ها با استفاده از سیاست SJF همواره کمینه خواهد بود.

۵-۸-۲ مدل‌های صف‌بندی

در بسیاری از سیستم‌ها، پردازه‌هایی که اجرا می‌شوند همه روزه تغییر می‌کنند، لذا مجموعه‌ی ثابتی از پردازه‌ها (یا زمان‌ها) وجود ندارد که بتوان از مدل‌سازی معین استفاده کرد. با این وجود، چیزی را که می‌توان معین کرد توزیع انفجارهای پردازنده و I/O می‌باشد. این توزیع‌ها را می‌توان اندازه‌گیری کرده و سپس تقریب یا به سادگی تخمین زد. حاصل این کار، یک فرمول ریاضی است که احتمال انفجار پردازنده‌ی خاصی را توصیف می‌کند. این توزیع در بیشتر موارد نمایی است و با میانگین آن بیان می‌شود. به طور مشابه، می‌توانیم توزیع زمان‌های ورود پردازه‌ها به سیستم را نیز به‌دست آوریم (توزیع زمان ورود). با این دو توزیعی که به‌دست می‌آید می‌توان برای بیشتر الگوریتم‌ها، متوسط بازده، بهره‌وری، زمان انتظار و موارد مشابه را محاسبه کرد.

سیستم کامپیوتری به صورت شبکه‌ای از کارگزارها توصیف می‌شود. هر کارگزار، صفی از پردازه‌های منتظر دارد. پردازنده کارگزاری برای صف آماده‌ی خود می‌باشد، همچنان که سیستم I/O کارگزار صف‌های دستگاه‌هاست. با دانستن نرخ‌های ورود و نرخ‌های سرویس می‌توانیم بهره‌وری، طول متوسط صف، میانگین زمان انتظار و موارد مشابه را محاسبه کنیم. این حوزه از مطالعه، **تحلیل شبکه‌ی صف‌بندی**[1] نامیده می‌شود.

به عنوان یک مثال، فرض کنید، n (به استثنای پردازه‌ای که سرویس می‌گیرد) متوسط طول یک صف بلندمدت، W میانگین زمان انتظار صف و λ متوسط نرخ ورود پردازه‌های جدید به صف (برای مثال، سه پردازه در هر ثانیه) باشند. انتظار داریم در طول زمان W که پردازه‌ای منتظر می‌ماند، $\lambda \times W$ پردازه‌ی جدید به صف وارد شوند. اگر سیستم در حالت پایدار باشد، در این صورت تعداد پردازه‌هایی که صف را ترک می‌کنند بایستی برابر تعداد پردازه‌هایی باشند که به صف وارد می‌شوند. بنابراین،

$$n = \lambda \times W$$

این معادله به **فرمول Little** معروف است که فوق‌العاده مفید است، چون برای هر الگوریتم زمانبندی و توزیع ورودی معتبر است. برای مثال، n می‌تواند تعداد مشتری‌ها در یک فروشگاه

1. Queueing-network analysis

باشد.

با استفاده از فرمول Little می‌توانیم با داشتن دو متغیر آن، یکی دیگر از سه متغیر را محاسبه کنیم. برای مثال، اگر بدانیم در هر ثانیه ۷ پردازه (به طور متوسط) وارد صف می‌شوند و ۱۴ پردازه معمولاً در صف وجود دارند، در این صورت می‌توانیم میانگین زمان انتظار هر پردازه را محاسبه کنیم که برابر ۲ ثانیه می‌شود.

تحلیل صف‌بندی می‌تواند در مقایسه‌ی الگوریتم‌ها مفید باشد، اما محدودیت‌هایی هم دارد. در حال حاضر، رده‌ی الگوریتم‌ها و توزیع‌هایی که می‌توان به‌دست آورد کاملاً محدود است. کار با ریاضیات الگوریتم‌ها و توزیع‌های پیچیده می‌تواند مشکل باشد. بنابراین، بیشتر توزیع‌های ورودی و سرویس، با روش‌های ساده‌ی ریاضی—اما غیرواقعی—تعریف می‌شوند. همچنین، به طور کلی لازم می‌شود که یک سری فرض‌های مستقلی انجام بگیرند که ممکن است درست نباشند. به عنوان پیامد این مشکلات، مدل‌های صف‌بندی در بیشتر موارد فقط تقریبی از سیستم‌های واقعی هستند و صحت نتایج محاسبات آنها را می‌توان زیر سؤال برد.

۵-۸-۳ شبیه‌سازی

برای دستیابی به ارزیابی دقیق‌تری از الگوریتم‌های زمانبندی می‌توانیم از **شبیه‌سازی**[1] بهره بگیریم. اجرای شبیه‌سازی‌ها در واقع شامل برنامه‌نویسی یک مدل سیستم کامپیوتری است. ساختمان‌داده‌های نرم‌افزاری، مؤلفه‌های اصلی سیستم را به نمایش می‌گذارند. شبیه‌ساز متغیری دارد که معرف ساعت است؛ با افزایش مقدار این متغیر، شبیه‌ساز حالت سیستم را تغییر می‌دهد تا فعالیت‌های دستگاه‌ها، پردازه‌ها و زمانبند را منعکس کند. همچنان که شبیه‌سازی اجرا می‌شود، آمارهایی که کارایی الگوریتم را نشان می‌دهند، جمع‌آوری و چاپ می‌شوند.

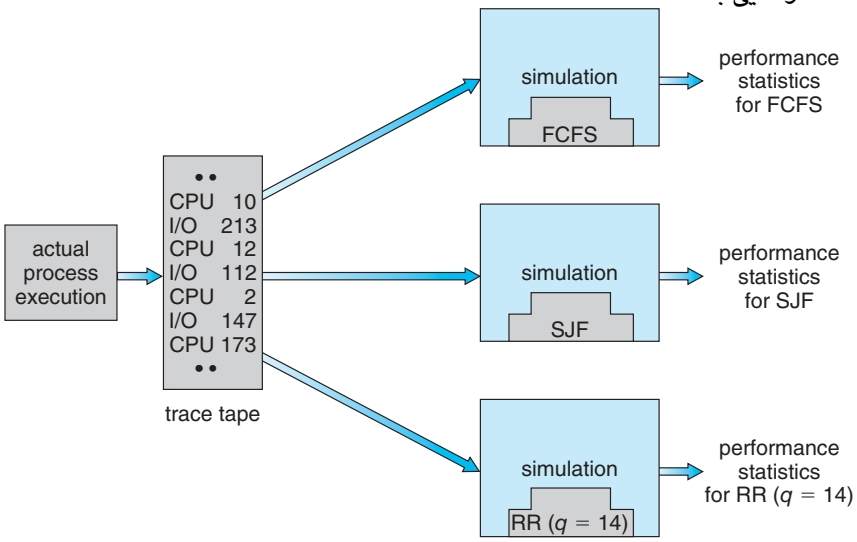
داده‌هایی که شبیه‌سازی را به پیش می‌رانند به شیوه‌های متعددی قابل تولید هستند. رایج‌ترین روش از یک مولد اعداد تصادفی استفاده می‌کند که برنامه‌نویسی می‌شود تا پردازه‌ها، زمان‌های انفجار پردازنده، ورودها، خروج‌ها و موارد مشابه را مطابق توزیع‌های احتمال تولید نماید. توزیع‌ها می‌توانند به صورت ریاضی (یکنواخت، نمایی، پواسن) یا تجربی تعریف شوند. اگر یک توزیع به صورت تجربی تعریف شود، بایستی پارامترهای سیستم واقعی تحت مطالعه مشخص شوند. نتایج به‌دست آمده، توزیع رویدادهای سیستم واقعی را تعریف می‌کنند؛ این توزیع می‌تواند برای راندن شبیه‌سازی استفاده شود.

با این وجود، یک شبیه‌سازی که با توزیع رانده می‌شود، ممکن است به دلیل وجود روابط میان رویدادهای پشت سر هم در سیستم واقعی، درست کار نکند. توزیع فرکانس فقط نشان می‌دهد که چه تعداد از نمونه‌های هر یک از رویدادها اتفاق می‌افتند؛ نمی‌تواند در رابطه با ترتیب آن رخدادها چیزی نشان بدهد. برای تصحیح این مسأله می‌توانیم از **فایل‌های ردیاب**[2] استفاده کنیم. ردیاب را با کنترل و نظارت بر سیستم واقعی و ثبت دنباله‌ی رویدادهای واقعی تهیه می‌کنیم

1. Simulation
2. Trace files

(شکل ۵-۳۱). سپس از این دنباله برای راندن شبیه‌سازی استفاده می‌کنیم. فایل‌های ردیاب روش ممتازی برای مقایسه‌ی دو الگوریتم با مجموعه‌ی یکسانی از ورودی‌های واقعی فراهم می‌کنند. این روش می‌تواند نتایج درستی برای ورودی‌های خود تولید کند.

شبیه‌سازی می‌تواند گران تمام شود، چون اغلب به ساعت‌های زیادی از زمان کامپیوتر نیاز دارد. با پرداختن به جزئیات بیشتر در شبیه‌سازی، نتایج دقیق‌تری حاصل می‌شود، اما باز هم به زمان بیشتری از کامپیوتر نیازمند است. به علاوه، فایل‌های ردیاب می‌توانند به حجم زیادی از فضای ذخیره‌سازی نیاز داشته باشند. بالاخره، طراحی، کدنویسی و عیب‌یابی شبیه‌ساز می‌تواند کار طاقت‌فرسایی باشد.



شکل ۵-۳۱ ارزیابی زمانبندهای پردازنده با شبیه‌سازی.

۵-۸-۴ پیاده‌سازی

حتی شبیه‌سازی نیز از دقت محدودی برخوردار است. تنها راه کامل و دقیق ارزیابی یک الگوریتم زمانبندی این است که آن را کد کرده و در یک سیستم‌عامل قرار دهیم و ببینیم که چگونه کار می‌کند. این روش، الگوریتم واقعی را برای ارزیابی در سیستم واقعی با شرایط عملیاتی حقیقی قرار می‌دهد.

این روش بدون هزینه نیست. هزینه از کدنویسی الگوریتم و تغییر سیستم‌عامل پشتیبان آن (به همراه ساختمان‌داده‌های مورد نیاز آن) به‌بار می‌آید. در آزمون تغییرات نیز هزینه وجود دارد که معمولاً در ماشین‌های مجازی است به‌جای این که روی سخت‌افزار اختصاصی باشد. آزمون رگرسیون تصدیق می‌کند که تغییرات، همه چیز را بدتر نمی‌کند، و باعث عیب‌های جدید یا تجدید دوباره‌ی عیب‌های قدیمی نمی‌شود (برای مثال، چون الگوریتم جایگزین شده عیبی را رفع کرده و تغییر آن باعث می‌شود آن عیب دوباره پیش بیاید).

مشکل دیگر این است که محیطی که الگوریتم در آن استفاده می‌شود تغییر خواهد کرد. این محیط نه تنها طبق معمول مطابق نوشتن برنامه‌های جدید و تغییر نوع مسائل عوض می‌شود، بلکه به خاطر کارایی زمانبند نیز تغییر می‌یابد. اگر الویت به پردازه‌های کوتاه داده شود، در این صورت کاربران امکان دارد پردازه‌های بزرگ‌تر خود را به مجموعه‌ای از پردازه‌های کوچکتر

بشکنند. اگر پردازه‌های محاوره‌ای نسبت به پردازه‌های غیرمحاوره‌ای الویت بگیرند، در این صورت ممکن است که کاربران سراغ پردازه‌های محاوره‌ای بروند. این مسئله معمولاً با استفاده از ابزار یا اسکریپت‌هایی که مجموعه‌های کاملی از عملیات را به‌صورت کپسول دارند، با استفاده‌ی مکرر از ابزار، و استفاده از ابزار در حالی که نتایج را اندازه‌گیری می‌کنند (و تشخیص هر مسئله‌ای که در محیط باعث می‌شوند).

البته، رفتار انسانی یا برنامه می‌تواند الگوریتم‌های زمان‌بندی را با چالش مواجه کند. برای مثال، محققان سیستمی طراحی کردند که با نگاه خودکار به حجم I/O در ترمینال، پردازه‌ها را به محاوره‌ای و غیرمحاوره‌ای دسته‌بندی می‌کرد. اگر پردازه‌ای در فاصله‌ی ۱ ثانیه‌ای، ورودی یا خروجی به ترمینال نداشت، در رده‌ی پردازه‌های غیرمحاوره‌ای قرار می‌گرفت و به یک صف با الویت پایین‌تر جابجا می‌شد. یک برنامه‌نویس برای مقابله با این سیاست، برنامه‌هایش را طوری تغییر داد که کاراکتر دلخواهی در فواصل منظم کمتر از ۱ ثانیه در ترمینال بنویسد. سیستم به برنامه‌های این برنامه‌نویس الویت بالا داد، هرچند که خروجی ترمینال کاملاً بی‌معنی بود.

در حالت کلی، انعطاف‌پذیرترین الگوریتم‌های زمانبندی آنهایی هستند که توسط مدیران سیستم یا کاربران قابل تغییر هستند، طوری که با یک برنامه یا مجموعه‌ای از برنامه‌های کاربردی خاص تطبیق یابند. برای نمونه، نیازهای زمانبندی یک ایستگاه‌کاری که با برنامه‌های گرافیکی کاربردی سطح بالا سر و کار دارد، در مقایسه با یک کارگزار وب یا یک کارگزار فایل متفاوت است. برخی سیستم‌عامل‌ها—به‌خصوص نسخه‌های متعددی از یونیکس—به مدیر سیستم این اجازه را می‌دهند که پارامترهای زمانبندی را مناسب پیکربندی یک سیستم خاص تنظیم نماید. برای مثال، سولاریس فرمان dispadmin را فراهم می‌کند تا مدیر سیستم بتواند پارامترهای رده‌های زمانبندی مطرح شده در بخش ۵-۷-۳ را تغییر بدهد.

رویکرد دیگر، استفاده از APIهایی است که الویت یک پردازه یا یک نخ را تغییر می‌دهند. APIهای جاوا، پوزیکس و ویندوز چنین توابعی را فراهم می‌کنند. نقطه ضعف این روش آن است که تغییر آهنگ کارایی یک سیستم یا یک برنامه‌ی کاربردی اکثراً موجب بهبود کارایی در شرایط کلی‌تر نمی‌شود.

۵-۹ خلاصه

- زمانبندی پردازنده، کار انتخاب پردازه‌ای منتظر از صف آماده و تخصیص پردازنده به آن می‌باشد. پردازنده توسط اعزام‌کننده در اختیار پردازه‌ی انتخابی قرار می‌گیرد.
- الگوریتم‌های زمان‌بندی می‌توانند پس‌گرفتنی (جایی که پردازنده می‌تواند از پردازه گرفته شود) و یا غیرقابل پس‌گیر باشند (جایی که یک پردازه بایستی داوطلبانه کنترل پردازنده را رها کند). تقریباً تمای سیستم‌عامل‌های مدرن پس‌گرفتنی هستند.
- الگوریتم‌های زمان‌بندی می‌توانند مطابق پنج معیار ارزیابی شوند که عبارتند از: (۱) بهره‌وری پردازنده، (۲) بازده، (۳) زمان گردش، (۴) زمان انتظار، و (۵) زمان پاسخ.